Technische Universität Braunschweig

A Branch-and-Bound Method for Bilevel Parallel Machine Scheduling with Adversarial Job Selection

Master Thesis Presentation

Ole Damke, April 2026
Institut für Mathematische Optimierung
Technische Universität Braunschweig

Gliederung

- Motivation & Problem
- Branch-and-Bound-Algorithmus
- Die Ceiling-Schranke
- Die Max-LPT-Schranke
- Frühzeitige Optimalitätserkennung
- Numerische Experimente
- Fazit

Motivation

Szenario: Produktionsbetrieb

- Manager entscheidet, welche Aufträge angenommen werden (Budgetbeschränkung)
- Planer weist Aufträge Maschinen zu — minimiert die Fertigstellungszeit
- Manager benötigt eine Worst-Case-Garantie für den Makespan

Dies ist ein **zweistufiges Optimierungsproblem**:

- **Leader** (Manager): wählt Aufträge zur Maximierung des Makespans
- **Follower** (Planer): minimiert den Makespan

Unser Problem

Obere Ebene (Leader)
Unbeschränktes Knapsack-Problem

↓ Auftragsportfolio

Untere Ebene (Follower)
Paralleles Maschinenplanungsproblem ($P||C_{max}$)

Zweistufige Optimierung: Definition

Zweistufiges Optimierungsproblem

$$\max_{x} \quad F\big(x,\ y^*(x)\big) \qquad\qquad x \in X$$

$$\text{wobei} \quad y^*(x) \in \arg\min_{y} \{f(x, y) : y \in Y(x)\}$$

- **Leader** wählt x unter Antizipation der optimalen Reaktion des Followers
- **Follower** beobachtet x und minimiert sein eigenes Ziel
- **Optimistisch:** bei Indifferenz des Followers wird das für den Leader günstigste Ergebnis gewählt

Zweistufige Optimierung: Schwierigkeit

Rechenkomplexität

Zweistufige Optimierung ist im Allgemeinen NP-schwer [Frangioni 1995].

Unsere spezifische Herausforderung

- **Leader** löst ein unbeschränktes Knapsack-Problem (NP-schwer)
- **Follower** löst paralleles Maschinenplanungsproblem $P||C_{max}$ (NP-schwer)
- Wertfunktion $C^*_{max}(x)$ ist nicht-konvex und unstetig. Kein geschlossener Ausdruck für $C^*_{max}(x)$: jede Auswertung erfordert das Lösen des Follower-Problems, was kombinatorische Verfahren (z. B. Branch-and-Bound) auf der oberen Ebene notwendig macht

Konsequenz

Allgemeine zweistufige Löser (z. B. MibS) können nur sehr kleine Instanzen verarbeiten. Wir benötigen einen **problemspezifischen exakten Algorithmus**.

Das Knapsack-Scheduling-Problem

Problemdaten:

- n Auftragstypen, m identische Parallelmaschinen, Budget $B > 0$
- Auftragstyp i: Bearbeitungszeit $d_i > 0$, Kosten $p_i > 0$; oBdA $d_1 \geqslant \cdots \geqslant d_n$
- **Leader** wählt $x_i \in \mathbb{Z}_+$ Kopien von Typ i; **Follower** weist $y_{ik} \in \mathbb{Z}_+$ Aufträge von Typ i Maschine k zu

Vollständige zweistufige Formulierung

$$\begin{aligned} \max_{x \in \mathbb{Z}_+^n} \quad & C^*_{max}(x) \\ \text{s.t.} \quad & \textstyle\sum_{i=1}^n p_i x_i \leqslant B \end{aligned}$$

$$\text{wobei} \quad C^*_{max}(x) = \min_{y_{ik} \in \mathbb{Z}_+} \max_{k=1,\dots,m} \sum_{i=1}^n d_i y_{ik}$$

$$\text{s.t.} \quad \textstyle\sum_{k=1}^m y_{ik} = x_i \quad \forall\, i \in \{1, \dots, n\}$$

Leader **maximiert** $C^*_{max}(x)$; Follower **minimiert** es — eine adversarielle zweistufige Struktur.

Warum Enumeration Scheitert

Naiver Ansatz: alle zulässigen x aufzählen und für jedes das Follower-Problem lösen.

$$\text{Suchraum} \leqslant \prod_{i=1}^{n} \left(\left\lfloor \frac{B}{p_i} \right\rfloor + 1 \right)$$

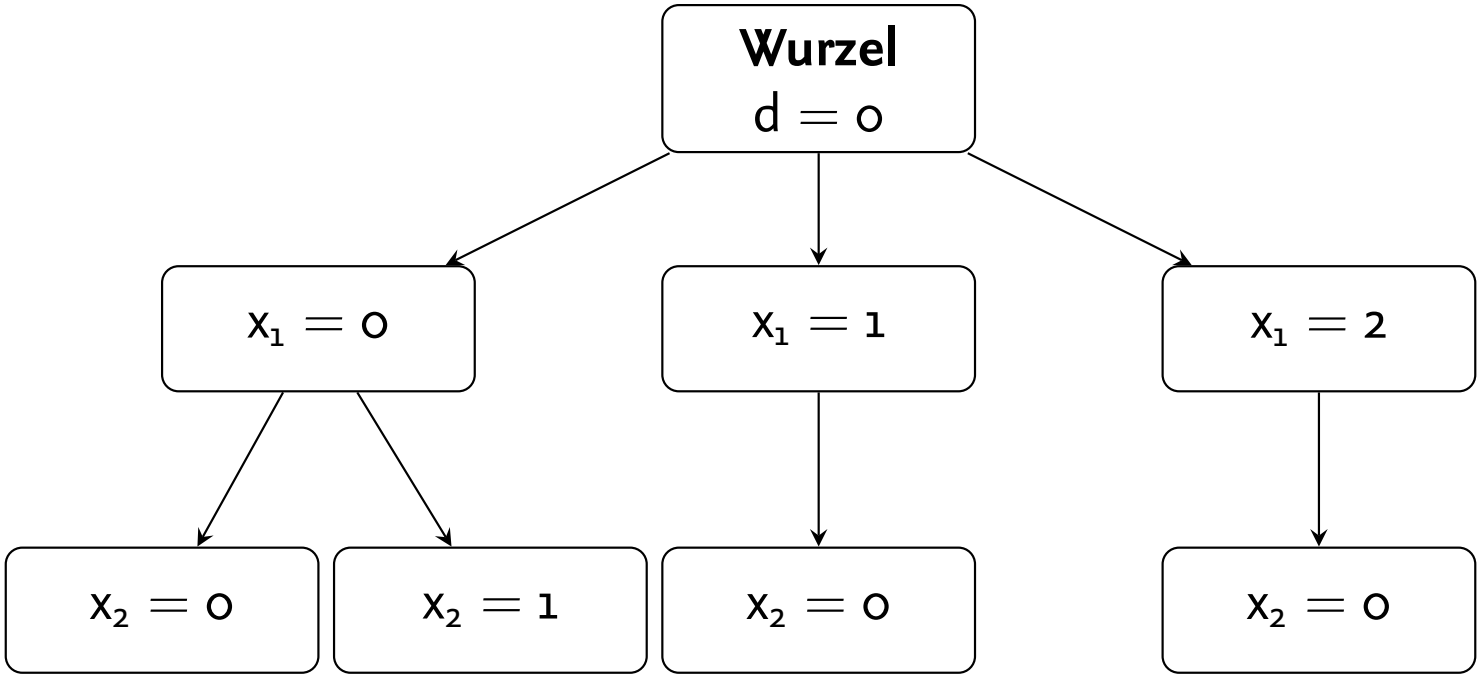
Beispiel: $n = 10$, $B = 100$, $p_i = 5+i$

Obere Schranke: $\mathbf{5{,}29 \times 10^{10}}$ Kandidaten

Bei 1 ms pro Follower-Lösung: $\approx \mathbf{1{,}68}$ Jahre

Lösung: Branch-and-Bound mit engen oberen Schranken zur aggressiven Baumbereinigung.

Suchbaumstruktur



Beispiel: $n = 2$ Typen, $p_1 = 5$, $p_2 = 8$, $B = 10$

Jedes **Blatt** erfordert eine Follower-Lösung (Scheduling). Bei großem n und B hat der Baum Milliarden von Blättern $\Rightarrow$ nicht handhabbar.

Branch-and-Bound: Überblick

Knotenstruktur auf Tiefe d

- **Fixierte Entscheidungen:** $x_1, \ldots, x_d$ $\quad B_{\text{rest}} = B - \sum_{i=1}^{d} p_i x_i$
- **Maschinenlasten:** inkrementeller LPT-Plan für Elemente $1, \ldots, d$
- **Kinder:** $x_{d+1} = 0, 1, \ldots, \lfloor B_{\text{rest}} / p_{d+1} \rfloor$

Drei Komponenten:

1. **Branching** — fixiere x_d
2. **Bounding** — berechne $\mathrm{UB}(d, B_{\text{rest}})$
3. **Pruning** — verwerfe wenn $\mathrm{UB} \leqslant C_{\max}^{\text{ink}}$

Designentscheidungen:

- Tiefensuche $\rightarrow$ schnelle Kandidatenaktualisierung $\rightarrow$ aggressives pruning
- Inkrementelles LPT: $O(m)$ pro Knoten
- DP vorberechnet: $O(1)$ Schrankenabfragen

Die LPT-Regel — Heuristik des Followers

Longest Processing Time (LPT) [Graham 1969]:

- Aufträge nach nicht-aufsteigender Dauer sortieren
- Jeden Auftrag der am wenigsten belasteten Maschine zuweisen

Garantie: $C^*_{\max}(x) \leqslant C^{\mathrm{LPT}}_{\max}(x) \leqslant \frac{4}{3} C^*_{\max}(x)$

Inkrementelle Aktualisierungen

Verzweigung $d \to d{+}1$: x_{d+1} Aufträge der Dauer d_{d+1} anhängen. Bestehenden sortierten Maschinenbelastungszustand wiederverwenden — **keine vollständige Neuberechnung**

LPT-Algorithmus

$L_1 = \cdots = L_m \leftarrow 0$
für jeden Auftrag j (nicht-aufsteigende Reihenfolge):
  $k^* \leftarrow \arg\min_k L_k$
  $L_{k^*} \leftarrow L_{k^*} + d_j$
return $C^{\mathrm{LPT}}_{\max} = \max_k L_k$

Zeit: $O(J \log m)$

Praxis: LPT ist in $\approx 84,6\%$ unserer Testinstanzen optimal.

Beschneiden durch Obere Schranken

Auf Tiefe d mit Budget B_{rest} berechnen wir eine obere Schranke:

$$C^*_{\max}(x_1, \ldots, x_n) \leqslant \mathrm{UB}(x_1, \ldots, x_d, B_{\text{rest}}) \quad \forall\, x_{d+1}, \ldots, x_n \text{ (Ergänzungen)}$$

prunen, wenn $\mathrm{UB} \leqslant C^{\text{ink}}_{\max}$.

$$\mathrm{UB} = \underbrace{C^{\mathrm{LPT},(d)}_{\max}}_{\text{fixiert, } 1..d} + \max_{\sum_{i>d} p_i y_i \leqslant B_{\text{rest}}} \underbrace{\Delta(y_{d+1}, \ldots, y_n)}_{\text{Mehraufwand Restelemente}}$$

Ceiling-Schranke

$\Delta = \sum_{i>d} d_i \lceil y_i / m \rceil$

Einzelne DP-Tabelle, $O(1)$ Abfrage

Max-LPT-Schranke

$\Delta = C^{\mathrm{LPT}}_{\max}(y_{d+1}, \ldots, y_n)$

DP-Tabelle pro Tiefe, engere Schranke

Ceiling-Schranke: Formulierung

Knapsack-Scheduling-Relaxierung

$$\mathrm{UB} = C_{\max}^{\mathrm{LPT},(d)} + \max_{y \geqslant 0} \sum_{i=d+1}^{n} d_i \left\lceil \frac{y_i}{m} \right\rceil \quad \text{s.t.} \sum_{i>d} p_i y_i \leqslant B_{\mathrm{rest}}$$

Intuition: $\lceil y_i/m \rceil$ Aufträge vom Typ i auf m gleichwertigen Maschinen erhöhen den Makespan um $\lceil y_i/m \rceil \cdot d_i$ — stets eine Überschätzung.

DP-Implementierung — O(1) Abfrage pro Knoten

Reduktion auf unbeschränktes Knapsack-Problem. Für Elementtyp i:

- **Einzel:** Kosten p_i, Wert d_i (für $1, \ldots, m-1$ Kopien)
- **Block:** Kosten $m \cdot p_i$, Wert d_i (für eine Charge von m)

Vorberechnung in umgekehrter Elementreihenfolge $\Rightarrow$ jede Tiefe-d-Abfrage ist ein direkter Tabellenlesezugriff.

Ceiling-Schranke: Korrektheitsbeweis

Satz

Für jede zulässige Vervollständigung gilt: $C^*_{max}(x_1, \ldots, x_n) \leqslant UB$

Beweisschema:

1. $C^*_{max}(x) \leqslant C^{LPT}_{max}(x)$ (LPT überschätzt)
2. Nach fixierten Elementen ist der aktuelle Makespan $= C^{LPT,(d)}_{max}$
3. Annahme: alle Maschinen bis $C^{LPT,(d)}_{max}$ gefüllt — nur Überschätzung
4. Auf m gleichmäßig belasteten Maschinen erhöht das Hinzufügen von y_i Aufträgen vom Typ i den Makespan um $\lceil y_i/m \rceil \cdot d_i$
5. Daher $C^{LPT}_{max}(x) \leqslant C^{LPT,(d)}_{max} + \sum_{i>d} \lceil x_i/m \rceil \cdot d_i$
6. Maximierung der rechten Seite über B_{rest} ergibt $UB \geqslant C^*_{max}(x)$ $\square$

Das Max-LPT-Problem

Definition

Finde $x \in \mathbb{Z}_+^n$ mit $\sum_i p_i x_i \leqslant B$, das $C_{\max}^{\mathrm{LPT}}(x)$ **maximiert.**

Zwei Rollen:

1. Einmalig gelöst $\Rightarrow$ **starker anfänglicher Kandidat** (mit $\frac{3}{4}$-Approximationsgarantie)
2. Für Restelemente an jedem Knoten gelöst $\Rightarrow$ **engere Schranke** als Ceiling

Berücksichtigt die tatsächliche LPT-Struktur (nicht nur $\lceil \cdot / m \rceil$).

Hauptergebnis

Max-LPT **optimal** in $O(n \cdot B)$ per DP gelöst.

Kompromiss

n tiefenspezifische DP-Tabellen vs. eine Ceiling-Tabelle; amortisiert durch besseres pruning.

Approximationsgarantie: Bausteine

Lemma (Eigenschaften der α-Approximation)

Sei $\texttt{approx}$ eine α-Approximation für Minimierung ($\alpha \geqslant 1$), setze $\beta := 1/\alpha$. Dann gilt für alle x:

$$\beta \cdot \mathrm{Opt}(x) \;\leqslant\; \beta \cdot \mathrm{approx}(x) \;\leqslant\; \mathrm{Opt}(x) \;\leqslant\; \mathrm{approx}(x)$$

Satz (Zweistufige Approx. [Weninger & Fukasawa 2025])

Wenn $\beta \leqslant 1$, $\beta \cdot \mathrm{Opt}(x) \leqslant \beta \cdot \mathrm{approx}(x) \leqslant \mathrm{Opt}(x) \leqslant \mathrm{approx}(x)$ für alle x, $\mathrm{approx}(x)$ für das Follower-problem stets zulässig ist und $\hat{x} \in \arg\max_x \mathrm{approx}(x)$, dann:

$$\beta \cdot \mathrm{Opt}_{\text{zweistufig}} \;\leqslant\; \mathrm{Opt}(\hat{x}) \;\leqslant\; \mathrm{Opt}_{\text{zweistufig}}$$

Unser Setting: LPT ist eine $\frac{4}{3}$-Approximation für $P||C_{max} \Rightarrow \beta = \frac{3}{4}$. Max-LPT maximiert $C_{max}^{LPT} \Rightarrow$ Satz direkt anwenden.

Die $\frac{3}{4}$-Approximationsgarantie

Satz

Sei $\hat{x}$ die Max-LPT-Lösung und x^* die zweistufig-optimale Lösung. Dann gilt:

$$\frac{3}{4}\, C^*_{max}(x^*) \;\leqslant\; C^*_{max}(\hat{x}) \;\leqslant\; C^*_{max}(x^*)$$

Beweis (unter Verwendung der zwei Bausteine):

1. **Lemma** ($\alpha = \frac{4}{3}$, $\beta = \frac{3}{4}$): für alle x, $\frac{3}{4}\, C^*_{max}(x) \leqslant \frac{3}{4}\, C^{LPT}_{max}(x) \leqslant C^*_{max}(x) \leqslant C^{LPT}_{max}(x)$
2. $\hat{x}$ maximiert C^{LPT}_{max}, also $C^{LPT}_{max}(x^*) \leqslant C^{LPT}_{max}(\hat{x})$
3. **Satz [WF25]:** $\frac{3}{4} \cdot \mathrm{Opt}_{zweistufig} \leqslant C^*_{max}(\hat{x}) \leqslant \mathrm{Opt}_{zweistufig}$
4. **Vollständige Kette:** $\frac{3}{4}\, C^*_{max}(x^*) \leqslant \frac{3}{4}\, C^{LPT}_{max}(x^*) \leqslant \frac{3}{4}\, C^{LPT}_{max}(\hat{x}) \leqslant C^*_{max}(\hat{x}) \leqslant C^*_{max}(x^*)$ $\square$

Praktische Bedeutung

Der Max-LPT-Kandidat ist **garantiert** $\leqslant 25\%$ **suboptimal**. In der Praxis ist die Lücke praktisch immer null (LPT ist oft optimal).

Max-LPT-Algorithmus: Budget pro Maschine & Eigenschaften

Strukturelle Beobachtung

In jedem LPT-Plan wurde der **zuletzt bearbeitete Auftrag** I auf die **am wenigsten belastete Maschine** zum Planungszeitpunkt gelegt:

$$C_{max}^{LPT} = C_{max}^{LPT(m)} + d_I$$

Der Makespan wird durch die minimale Maschinenlast unmittelbar vor I plus seiner Dauer d_I bestimmt.

Für einen festen Kandidaten-Letzten-Auftrag J wollen wir $C_{max}^{LPT(m)}$ für Elemente $1, \ldots, J$ maximieren.

Zwei Haupteigenschaften des optimalen vorhergehenden Plans

- **Eigenschaft 1** (höchstes Budget pro Maschine):
  $B^* := \lfloor B/m \rfloor$ ist das größte Budget, das gleichmäßig jeder Maschine zugewiesen werden kann.
  Keine Umverteilung kann der am wenigsten belasteten Maschine mehr als B^* geben.
- **Eigenschaft 2** (beste Elemente für B^*):
  Für Budget B^* ist die optimale Elementkombination aus Typen $\leqslant J$ das Optimum des unbeschränkten Knapsack-Problems.

Wenn beide gelten, erzielt kein Plan ein größeres $C_{max}^{LPT(m)}$: kaufe m Kopien jedes DP-optimalen Elements und führe LPT aus.

Budgetanpassung wenn $\hat{B} := B - mB^* < p_J$

Rest $\hat{B}$ ist zu klein, um eine Kopie von J zu bezahlen. Reduziere B^* um 1 (gleichmäßig über Maschinen) bis $\hat{B} \geqslant p_J$. Eigenschaft 1 gilt für das reduzierte B^*; frage $\mathrm{DP}(J, B^*)$ erneut ab. Wenn B^* vor $\hat{B} \geqslant p_J$ null erreicht, überspringe J (unzulässig).

Max-LPT-Algorithmus: Korrektheit & Komplexität

Algorithmus (Pseudocode)

DP-Tabelle vorberechnen (unbeschränktes Knapsack-Problem, Elemente $1 \dots n$, Budgets $0 \dots B$).

$z^* \leftarrow 0$ // bisher bester Kandidat

für jeden Typ $J \in \{1, \dots, n\}$:

  $B^* \leftarrow \lfloor B/m \rfloor$

  solange $(B - mB^*) < p_J$ **und** $B^* > 0$: $B^* \leftarrow B^* - 1$

  $z_J \leftarrow \mathrm{DP}(J, B^*) + d_J$; $z^* \leftarrow \max(z^*, z_J)$

return z^*

Korrektheit

Jede optimale Lösung hat einen letzten Auftrag I.

Für dieses I identifizieren Eigenschaften 1 & 2 einen optimalen vorhergehenden Plan via $\mathrm{DP}(I, B^*)$.

$\max_J z_J$ über alle Kandidaten deckt das Optimum ab. □

Komplexität: $O(n \cdot B)$

DP-Tabelle in $O(n \cdot B)$ vorberechnet.

Pro Kandidat J: $O(1)$ Abfrage + $O(B^*/m)$ Anpassung.

n Kandidaten $\Rightarrow O(n \cdot B)$ gesamt.

Frühzeitige Optimalitätserkennung

Schlüsselungleichung

Für alle x: $C^*_{max}(x) \leqslant C^{LPT}_{max}(x)$ (LPT überschätzt nur)

Sei $\hat{x} \in \arg\max_x C^{LPT}_{max}(x)$. Wenn $C^{LPT}_{max}(\hat{x}) = C^*_{max}(\hat{x})$, dann gilt für alle zulässigen x:

$$C^*_{max}(\hat{x}) = C^{LPT}_{max}(\hat{x}) \geqslant C^{LPT}_{max}(x) \geqslant C^*_{max}(x)$$

$\Rightarrow$ $\hat{x}$ ist das **globale zweistufige Optimum** - kein B&B-Baum erforderlich!

Auswirkung

- $C^*_{max}(\hat{x})$ wird ohnehin berechnet, um den Kandidaten zu initialisieren
- Diese Überprüfung kostet **nichts extra**
- In $\approx$ **84,6**% unserer Testinstanzen ausgelöst $\Rightarrow$ an der Wurzel gelöst

Complete Algorithm

1. Solve Max-LPT $\rightarrow \hat{x}$; compute $C_{max}^{LPT}(\hat{x})$, $C_{max}^{*}(\hat{x})$
2. **If** $C_{max}^{LPT}(\hat{x}) = C_{max}^{*}(\hat{x})$: **return** $\hat{x}$
3. $C_{max}^{ink} \leftarrow C_{max}^{*}(\hat{x})$; push root to stack Q
4. **While** $Q \neq \emptyset$:
   - Remove node $(d, x_1, \ldots, x_d, B_{rest})$
   - If $d = n$: solve follower; update C_{max}^{ink}
   - Else: compute $UB(d, B_{rest})$
   - If $UB \leqslant C_{max}^{ink}$: **prune**
   - Else: push children (depth-first)
5. Return incumbent

Inspired by [Weninger & Fukasawa 2025].

Leaf optimisation

At depth $n-1$: only the child spending maximum remaining budget matters. All others produce a smaller or equal C_{max}^{*}.

Follower solver

Gurobi 12.0.3 [Gurobi Optimization 2024] solves the MIP formulation of $P||C_{max}$ at leaves.

Versuchsaufbau

Hard- & Software:

- AMD Ryzen 5 9600X, 32 GB RAM
- Python 3.11, Gurobi 12.0.3
- Einzelthreadige Läufe

PatternTestSet: 140 Instanzen

7 strukturelle Muster:
Uniform Ratios, High Variance, Increasing, Moderately Random, Extreme, Strong Correlation, Subset Sum

6–12 Typen, 3–8 Maschinen, Budget 15–50

Korrektheit: vs. Enumeration

- 140 Instanzen: **0 Abweichungen**
- Enumeration bei 15/140 Zeitlimit überschritten
- B&B löste alle 140 in ms

Beschleunigung

Bis zu $\mathbf{10^6} \times$ schneller als vollständige Enumeration

Gitternetz-Sensitivitätsanalyse

Gitter: 6×6 (Typen × Maschinen), 3 Budgetmultiplikatoren, 5 Seeds = 540 Instanzen; 1 h Limit.

Typen: 4, 8, 12, 16, 20, 24 • **Maschinen:** 2, 4, 6, 8, 10, 12 • **Multiplikatoren:** 1,3; 2,5; 5,0

Budget ×	Wurzel opt.	Fertiggest.	Zeitüberschr.	Blatt Zeitü.
1,3×	154 / 154	22 / 26	2 / 0	2 / 0
2,5×	150 / 150	19 / 22	3 / 0	8 / 8
5,0×	153 / 153	20 / 23	4 / 1	3 / 3

Ceiling / MaxLPT

Hauptergebnis

84,6% der Instanzen an der Wurzel gelöst — kein B&B-Baum erforderlich

MaxLPT-Vorteil

Mehr Fertigstellungen und weniger Zeitüberschreitungen in jeder Konfiguration

Ceiling vs. MaxLPT: Schwierige Instanzen

61 nicht-wurzeloptimale Instanzen, bei denen beide Schranken innerhalb von 1 h fertiggestellt wurden:

Laufzeit (Ceiling − MaxLPT):

- Mittelwert: +71 s (Ceiling langsamer)
- Median: +0,4 s
- Max: +1.057 s
- MaxLPT schneller: **41 von 61 Fällen**

Knoten (Ceiling − MaxLPT):

- Mittelwert: +4,8 Mio. Knoten
- Median: +69 K
- Max: +54,5 Mio.
- MaxLPT immer weniger Knoten

Interpretation

MaxLPT beschneidet aggressiver bei schwierigen Instanzen. Die Vorberechnung von n DP-Tabellen ist durch die reduzierte Knotenanzahl vollständig gerechtfertigt. Ceiling ist marginal schneller bei trivialen (wurzeloptimalen) Fällen.

Vergleich mit MibS

MibS: allgemeiner zweistufiger MILP-Löser [Tahernejad et al. 2020]
Teilmenge: $m \in \{4, 8, 12\}$, $j \in \{4, 8, 16, 20\}$, 3 Multiplikatoren, 5 Seeds $= 180$ Instanzen

m \ j	4	8	16	20
4	15/15	8/15	2/15	1/15
8	5/15	0/15	0/15	1/15
12	2/15	0/15	0/15	0/15

MibS-Fertigstellungen (von 15) pro Zelle. MaxLPT B&B: **179/180**.

Beschleunigung gegenüber MibS (1 h Limit)

Ceiling: Mittelwert +2.876,6 s, Median +3.599,9 s MaxLPT: Mittelwert +2.959,6 s, Median +3.599,5 s
Probleme, die unser Algorithmus in Millisekunden löst, benötigen bei MibS **über eine Stunde**.

Zusammenfassung & Schlussfolgerungen

Beiträge:

1. Formales zweistufiges Rucksack-Scheduling-Modell
2. Ceiling-Schranke mit vorberechnetem $O(1)$ DP, lösbar in $O(n \cdot B)$
3. Max-LPT optimal in $O(n \cdot B)$ gelöst
4. $\frac{3}{4}$-Approximationsgarantie (Theorie)
5. Frühzeitige Optimalitätserkennung (84,6% an Wurzel gelöst)
6. Umfassende numerische Studie

Übertrifft MibS um **Größenordnungen**.

Einschränkungen

- Keine Charakterisierung einfacher Instanzen
- Nur identische Maschinen

Zukünftige Arbeit

- Verwandte/unverwandte Maschinen
- Wurzeloptimale Instanzen charakterisieren

Vielen Dank!

Fragen?

Ole Damke • o.damke@tu-braunschweig.de

Institut für Mathematische Optimierung • TU Braunschweig

Hinweis zur KI-Nutzung

Einsatz KI-basierter Hilfsmittel

Teile dieses Vortrags wurden mithilfe von KI-gestützten Werkzeugen (GitHub Copilot (GPT-4.1), Anthropic Claude (Claude Opus 4.6, Claude Sonnet 4, Claude Sonnet 4.6)) generiert, überarbeitet und/oder verbessert. Die KI wurde für verschiedene Aufgaben eingesetzt, darunter das Erstellen von Grundversionen, das Überarbeiten der Folien und dem Beheben von Fehlern. Alle generierten oder modifizierten Abschnitte wurden von mir sorgfältig geprüft, nachvollzogen und ggf. angepasst. Die Verantwortung und das Verständnis für die hier präsentierten Inhalte liegen vollständig bei mir.

Literatur I

- A. Frangioni. On a new class of bilevel programming problems and its use for reformulating mixed integer problems. Eur. J. Oper. Res., 82(3):615–646, 1995.
- R. L. Graham. Bounds on Multiprocessor Timing Anomalies. SIAM J. Appl. Math., 17(2):416–429, 1969.
- N. Weninger, R. Fukasawa. A fast combinatorial algorithm for the bilevel knapsack problem. Math. Program., 210(1):847–879, 2025.
- S. Tahernejad, T. K. Ralphs, S. T. DeNegre. A branch-and-cut algorithm for mixed integer bilevel linear optimisation. Math. Program. Comput., 12(4):529–568, 2020.
- T. Kleinert et al. A survey on MIP techniques in bilevel optimisation. EURO J. Comput. Optim., 9:100007, 2021.
- S. Dempe, A. Zemkoho. Bilevel Optimization, vol. 161. Springer, 2020.

Literatur II

Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual. Version 12.0, 2024. https://www.gurobi.com